Practical – 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely? Yes. The session_start() function is used to start sessions securely. During logout, session data and cookies are cleared, and session_destroy() is used to destroy the session.

2. Is password_verify() used correctly? Yes. The password_verify() function compares the password entered during login with the hashed password stored in the database. This prevents the need to store or compare plain-text passwords.

3. Is unauthorized dashboard access blocked? Yes. The authentication check ensures that users must log in before accessing the dashboard. Role-based checks also prevent students from accessing the admin dashboard and vice versa.

4. Is session timeout implemented? Yes. A 15-minute inactivity timeout is implemented. If a user remains inactive for longer than 15 minutes, the session is destroyed and the user is redirected to the login page.

5. Are admin and student users redirected to appropriate dashboards? Yes. After successful login, users are redirected according to their assigned role. Admin users are sent to the admin dashboard, while student users are sent to the student dashboard.

B. Implementation/ Output/Result:

In this practical, a secure login and logout system was implemented for the StudentHub Portal using PHP, MySQL, and session management. The existing student registration system was extended to authenticate registered users and provide role-based access to the dashboard.

First, a login page was connected to the PHP backend to accept the Student ID and password. PHP retrieves the corresponding student record from the studenthub_db database and verifies the entered password against the stored hashed password using the password_verify() function. If the credentials are incorrect, an appropriate error message is displayed; otherwise, the user is successfully authenticated.

After successful authentication, a PHP session is started and the session ID is regenerated using session_regenerate_id(true) to improve security. The user's details, including Student ID, name, and role, are stored in session variables. Based on the assigned role, administrators are redirected to the admin dashboard, while students are redirected to the student dashboard. Authentication and role-based access checks prevent unauthorized users from accessing restricted dashboards.

Finally, a 15-minute session timeout mechanism is implemented to automatically log out inactive users. A separate logout script securely clears session variables, removes the session cookie, and destroys the session when the user clicks the Logout option. The implementation was integrated into the existing StudentHub Portal, and the contact page was modified to display static contact information instead of collecting user details.

C. Evidence of Work (Screenshots):

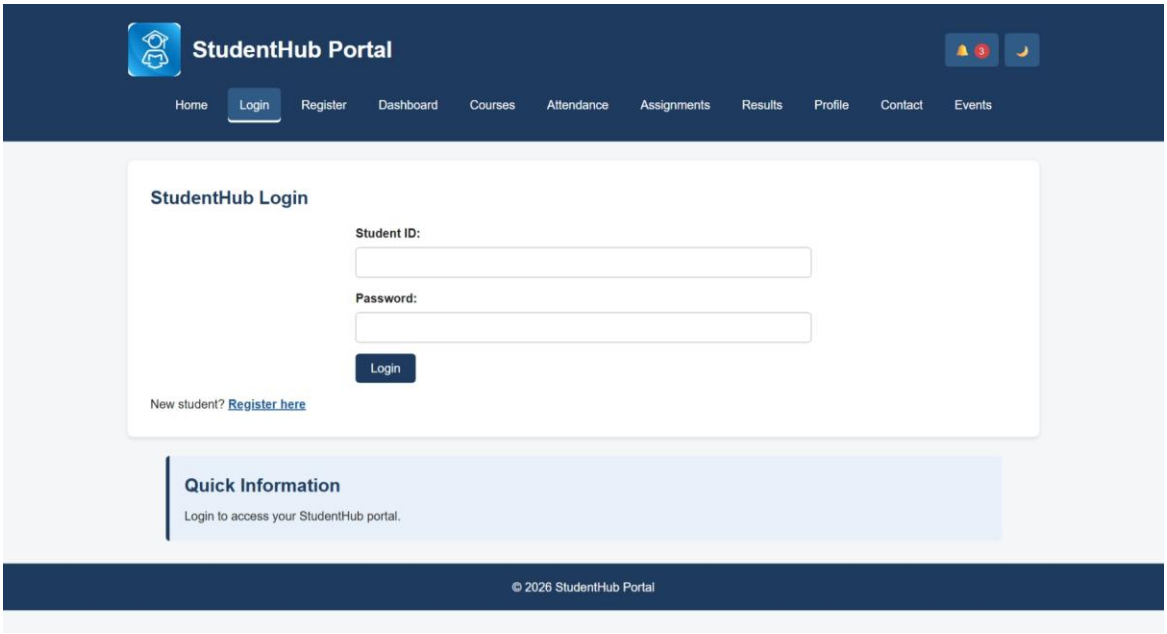


Fig1: Login Page

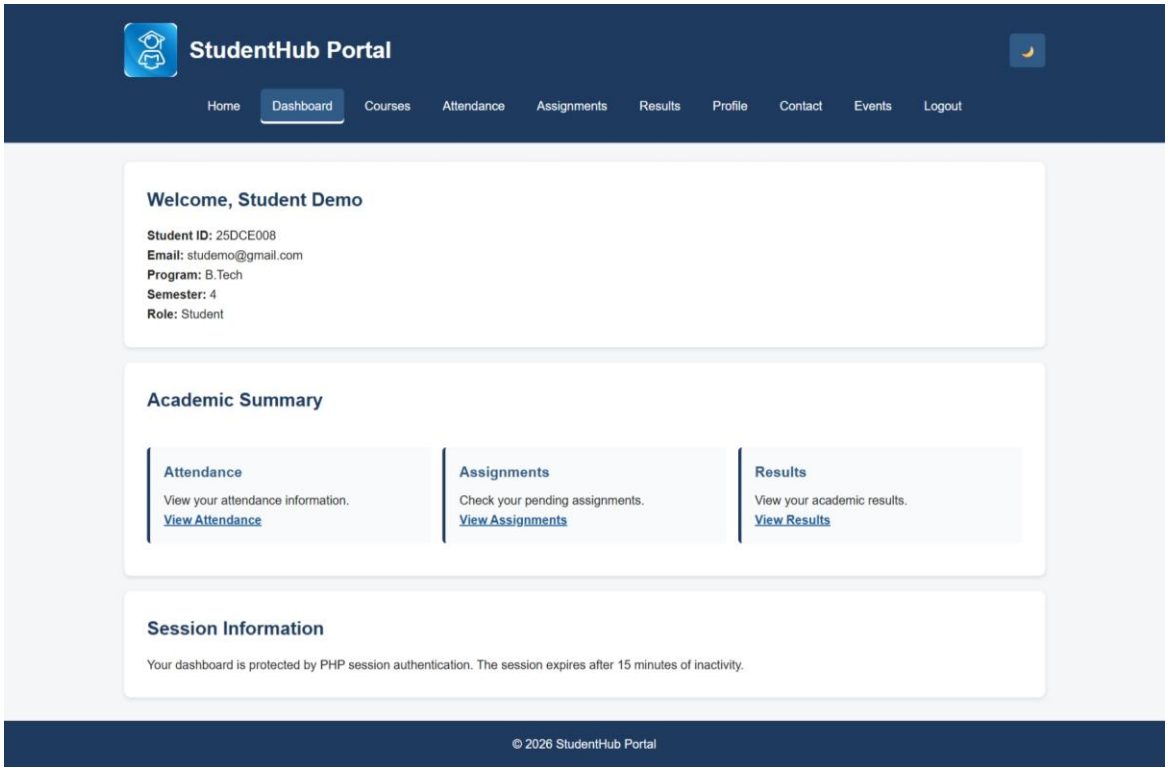


Fig2: Successful Login

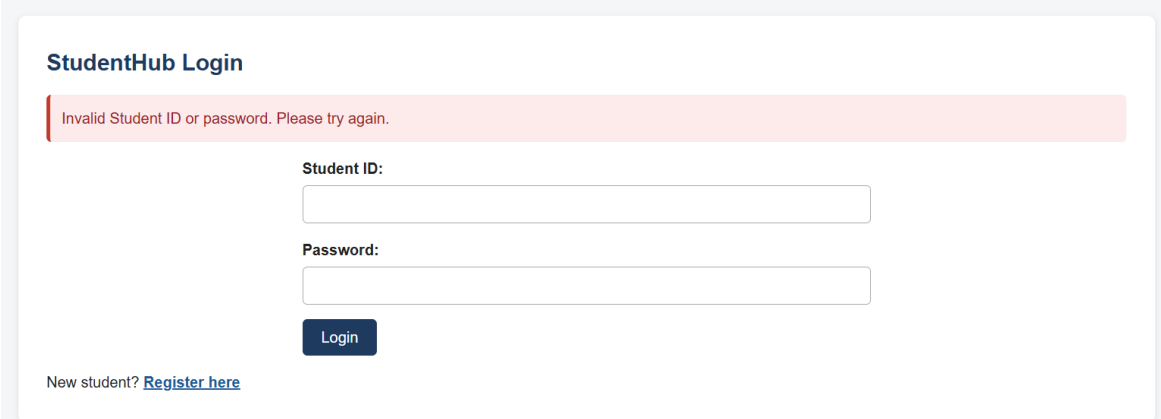


Fig3: Failed Login

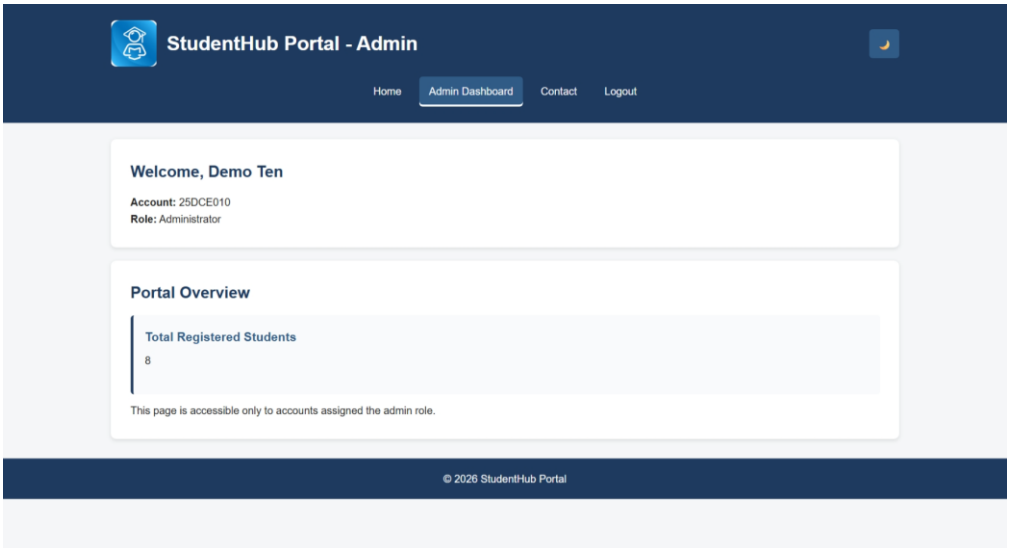


Fig4: Admin Login

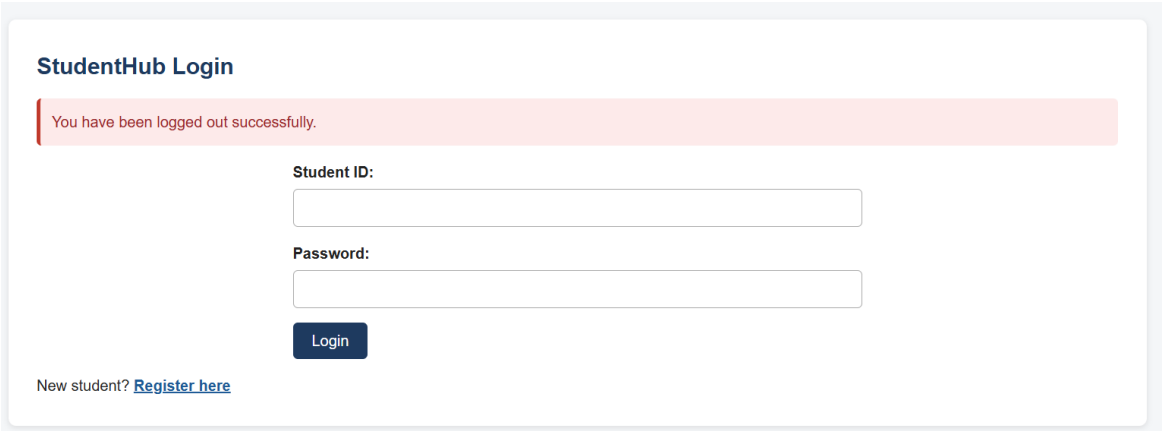


Fig5: Logout

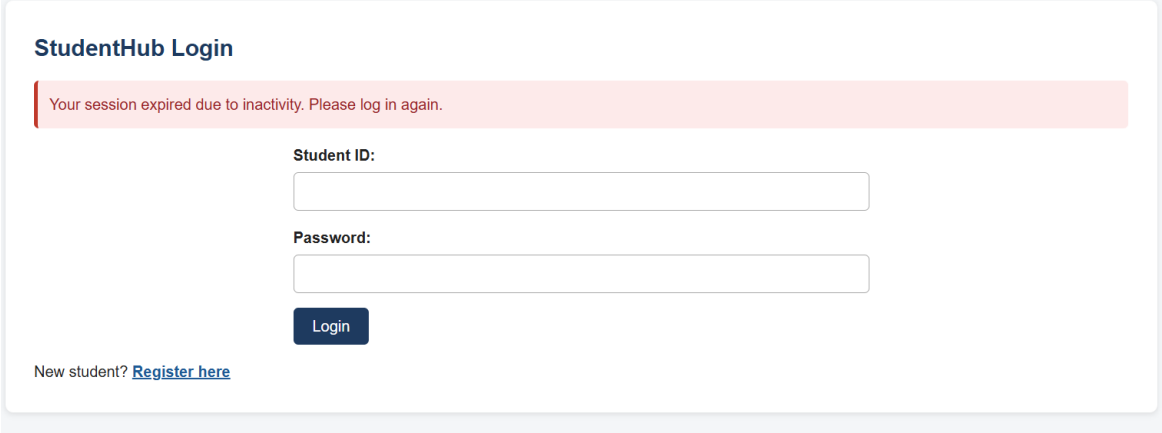


Fig6: Session Timeout

D. GitHub Repository Link:

aryan150806-wq/StudentHubPortal: Web Development Framework P

E. Challenges Faced:

1. Database Connectivity: Establishing a proper connection between PHP and the MySQL database to retrieve student records during login.

2. Password Verification: Implementing password_verify() correctly to authenticate users using their stored hashed passwords.

3. Session Management: Starting, maintaining, and securely destroying PHP sessions during login and logout.

4. Role-Based Access Control: Ensuring that students and administrators are redirected to their respective dashboards and cannot access unauthorized pages.

5. Session Timeout and Redirection: Implementing automatic logout after 15 minutes of inactivity and resolving issues with form actions and PHP page redirection in XAMPP.

F. Learning Outcomes:

Through this practical, I successfully implemented a secure login and logout system using PHP, MySQL, and session management. I learned to verify passwords securely using password_verify() and manage user sessions. I also implemented role-based access control for student and admin dashboards. Additionally, I understood how to restrict unauthorized access and automatically log out inactive users using session timeout.

Faculty Remarks: